



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

TinyML-enabled Human indoor localization with UWB-radar imaging

TESI DI LAUREA MAGISTRALE IN
COMPUTER SCIENCE AND ENGINEERING - INGEGNERIA INFORMATICA

Giampiero Repole, 968541

Advisor:
Prof. Manuel Roveri

Co-advisors:
Massimo Pavan
Pierpaolo Lento

Academic year:
2023-2024

Abstract: In the era of rapid technological evolution, the surge in embedded devices, edge computing, and IoT units is reshaping computing paradigms towards distributed and edge systems. TinyML, or Tiny Machine Learning, stands at the forefront of this transformation, by enabling AI capabilities on resource-constrained devices. Particularly crucial in IoT and edge computing environments, TinyML facilitates data processing close to the source to reduce latency, save energy, and maintain data privacy. Ultra-Wideband (UWB) radar is a particularly promising technology for a wide variety of applications such as localization, presence detection and communication, due to its large bandwidth, relatively low cost, low power consumption and robust performance, representing a cutting-edge solution that seamlessly integrates with TinyML principles. In this thesis we introduce for the first time in the literature a lightweight, privacy-preserving, TinyML solution for human indoor localization powered by Multiple-Input Multiple-Output UWB radar. To achieve this goal we propose a Pipeline that extracts Range-Angle maps from signals of multiple UWB receiver antennas and use them as input for a lightweight object detection network, adapted to work on radar data for human localization and counting purposes. The solution has been tested on multiple datasets with different UWB radar antennas configuration, analyzing the trade-off between different energy consumption levels and localization capability.

Key-words: UltraWideBand radar; Radar imaging; Tiny Machine Learning; Human Indoor Localization;

1. Introduction

Recent years have seen a rapid increase in the use of embedded systems and edge devices. This growth is driven by several factors, including advancements in technology, the proliferation of the Internet of Things (IoT), and the need for faster data processing and decision-making at the source of data generation, while preserving the privacy of the users. In this context, TinyML, or Tiny Machine Learning, aim to enable AI capabilities on resource-constrained devices offering several advantages such as reduced computation latency, energy saving, and privacy-preserving systems. Moreover, the integration of Ultra-Wideband (UWB) radar technology represents a cutting-edge solution that seamlessly integrates with TinyML principles. UWB radar technology leverages ultra-wideband radio waves to deliver precise sensing in compact devices while preserving privacy. These radar systems are designed for high precision in detecting environmental changes with high accuracy, for energy efficiency, with minimal power consumption, and fast data acquisition, which is crucial for real-time applications, making them a viable alternative to camera-based applications. The combination of advanced machine learning algorithms, through TinyML, and UWB radar technology unlocks the opportunity to develop innovative and efficient applications such as smart sensing, localization, and gesture recognition. This topic opens up a wide range of research areas, including people counting, human presence detection, position estimation or human indoor localization tasks, for various scenarios. In this thesis we explored the Multiple-Input Multiple-Output (MIMO) UWB radar sensing capability combined with TinyML object detection algorithms proposing a novel lightweight pipeline for human indoor localization tasks. In particular, the pipeline extracts a Range-Angle radar map from a MIMO UWB radar data through a *UWB radar imaging module* and use it to count and locate human targets in an indoor environment with a *Human Localization on Radar Maps (HLRM) module*. This solution represents a privacy-preserving, low power consumption and low latency alternative for tiny devices compared to camera-based localization systems, useful in scenarios where the focus is the privacy of the users, such as patient monitoring in hospitals or rest homes, automotive, or meeting rooms.

1.1. Thesis Structure

- **Section 2** provides the necessary background for the technologies used in this thesis. First, we introduce the characteristics of UWB radar systems and their imaging capabilities using different signal processing algorithms. We then give an overview of the object detection task in computer vision, and the recent advances in TinyML to bring it on constrained devices.
- **Section 3** presents the state of the art in human localization using UWB radar signals and some works on the application of TinyML algorithms to UWB radar data.
- **Section 4** contains the formal description of the human localization on radar maps problem analyzed in this thesis and the *Indoor Human 2D-Localization Pipeline* proposed as solution.
- **Section 5** describes the datasets and experimental settings used to test our solution. We also provide the characteristics of UWB radars used to collect the datasets.
- **Section 6** reports and compare the experimental results obtained in the different settings analyzed.
- **Section 7** draws the conclusions and presents the future directions on this work.

2. Background

In this section we aim to provide the necessary knowledge about the technologies involved in the design of our proposed solution.

Section 2.1 gives an overview of UWB radar technology and its positioning capabilities, formalizing the algorithms used for the radar imaging extraction.

Then *Section 2.2* gives an understanding of TinyML and its context of application, and *Section 2.3* describes the object detection task in computer vision and the implementation of a recent TinyML object detection model for constrained devices.

2.1. UWB Radar Imaging

2.1.1 Ultra-Wideband Radar

Ultra-wideband (UWB) radars are defined as a wireless technology which transmits sub-nanosecond pulses, that uses very low power radio waves to communicate over a short distance [5].

It is characterized by its use of a broad spectrum of frequencies, between 3.1 GHz and 10.6 GHz, and a bandwidth typically greater than 500MHz or at least 20% of the radar center frequency, in combination with very short narrow pulses.

UWB radar systems can be categorized into Direct Sequence Ultra Wideband (DS-UWB) and Impulse Radio Ultra Wideband (IR-UWB), distinguishing by how they transmit and receive signals.

DS-UWB radar operates through a direct-sequence spread spectrum technique, where a narrowband carrier signal is modulated by a wideband Pseudo-Random Noise (PN) code. This modulation spread the signal's energy across a broad frequency spectrum. The receiver decodes the signal by using the identical PN code to compress it back to its original bandwidth.

IR-UWB radar, instead, employs an impulse-based approach for signal transmission and reception, leveraging very short-duration pulses to achieve wideband transmission without the need for a continuous carrier signal. The receiver picks up these pulses and calculate the target's distance by measuring the Time of Flight (ToF).

IR-UWB is simpler to implement, uses less power, and is more robust against multipath interference[5], making it a superior option for battery-operated devices and in settings with many obstacles or potential interference sources, and we will refer to it as UWB radar in this thesis.

The wider bandwidth enables finer Range Resolution in the order of few centimeters making it highly effective for detailed imaging and precise distance measurements within a short range (10 – 15meters), where the Range Resolution is the radar ability to distinguish between 2 closely spaced targets and can be calculated as:

$$R_{res} = \frac{c}{2 \times B} \quad (1)$$

where c is the speed of light and B is the UWB radar bandwidth.

UWB radars are ideal for real-time location-based tasks, indoor ranging and positioning, radar target (stationary or moving) tracking, and in medical applications such as monitoring breathing and heart rate[5], under its sensing accuracy limitations.

The capacity of the UWB pulses to penetrate through obstacles and its resistance to multipath interference provides reliable performance in various demanding conditions, makes it exceptionally capable of detecting human movements, even through obstacles such as wall. These characteristics underline its growing importance in a wide range of applications focused on human localization and tracking.

2.1.2 Uniform Linear Array of Antennas

A Uniform Linear Array (ULA) consists of sensor elements, such as dipole antennas, equally spaced along a straight line. These arrays are utilized in various applications, including radar and communications, to improve signal reception and transmission. The key advantage of a ULA is its ability to form directional beams, enhancing signal quality by focusing on specific directions while minimizing interference from others. The spacing between the elements is crucial and is typically less than or equal to half the wavelength λ of the transmitted signal to avoid grating lobes, which are unwanted directions of high gain[2].

With multiple receiver antennas, a radar pulse reflecting off a target may arrive at one of the antennas later than the other. This time difference makes it possible to calculate the angle the reflection is coming from, meaning that we can combine distance information, provided by each receiver, with the angular information, to get an understanding of where the reflection is coming from in 2D-space.

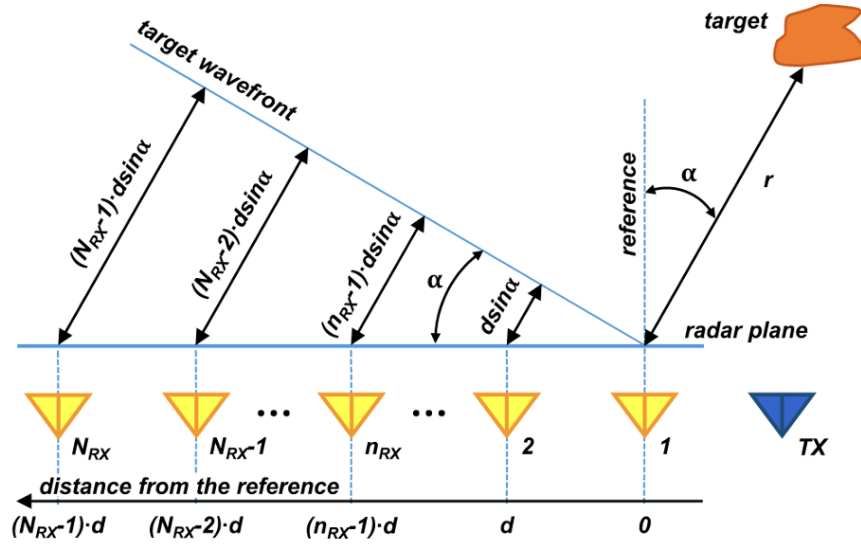


Figure 1: ULA of receiver antennas [16]

A generic Single-Input Multiple-Output (SIMO) UWB radar system can be model as a ULA of sensors, representing a collection of N_{RX} receiver antennas physically deployed on the UWB device, and equally spaced of δ_{RX} along a straight line, where transmitter antenna position is approximated with array's first element position.

Assuming antennas disposition like in Figure 1 and a target in the far field at a distance r and angle α from the first element of the array (considered as the reference), the signal reflected from the target should travel an additional distance of

$$\Delta_r = (n_{RX} - 1) \cdot \delta_{RX} \cdot \sin \alpha \quad (2)$$

to reach the n_{RX} -th antenna. This corresponds to a phase difference of

$$\Delta_\phi = \frac{2\pi\delta_{RX}}{\lambda} \cdot \sin \alpha \quad (3)$$

among the signals received by adjacent antennas, where λ is the wavelength of the transmitted radar signal[16].

2.1.3 Digital Beamforming

Digital Beamforming (DBF) is a signal processing technique in the time domain for enhancing signal-to-noise ratio of the signal from a specific direction, or Digital Beam, by delaying and summing the signals from an ULA of sensors. Digital beams are generated by calculating phase-shifts (i.e. delays), that will focus a digital beam to a given angle, for all the receiving antennas taking as reference the first element of the array. These delays can then be applied as weights to the ULA receivers and the resulting weighted signals can be summed together to form a digital beam at the given angle. More formally, having a list of angles $\Theta = \{\theta_1, \dots, \theta_{N_a}\}$ called Steering Vector, then we can compute the DBF weights $\forall \theta_i \in \Theta, \forall n_{RX} \in \{1, \dots, N_{RX}\}$ as:

$$W(\theta_i) = \exp \left[-j \frac{2\pi}{\lambda} \delta_{RX} (n_{RX} - 1) \sin \theta_i \right] \quad (4)$$

where λ is the wavelength of the transmitted radar pulse, δ_{RX} is the distance between ULA receiver antennas and j is the imaginary unit. The result of the juxtaposition of these vectors is the matrix $W \in \mathbb{C}^{N_a \times N_{RX}}$.

Let $s_n \in \mathbb{C}^K$ be the complex signal received by the n_{RX} -th element of the array, $n_{RX} \in \{1, \dots, N_{RX}\}$, where $K \in \mathbb{N}$ is the number of quantized distances in the radar acquisition range, also called Range profile. The radar imaging algorithm takes as input the set of radar acquisitions from N_{RX} antennas, $S \in \mathbb{C}^{N_{RX} \times K}$, and map them to a Range-Angle map $S_{angle} \in \mathbb{R}^{N_a \times K}$, that contains magnitudes of the acquired signals in the angle directions $\Theta = \{\theta_1, \dots, \theta_{N_a}\}$ and range profile K , for a single radar frame.

2.1.4 Fast-Fourier Transform

The Fast Fourier Transform (FFT) is an optimized algorithm for computing the Discrete Fourier Transform (DFT) and its inverse. It transforms a signal from its original domain (often time or space) into a representation in the frequency domain and vice versa. The FFT significantly reduces the computational complexity of performing a DFT from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$ where n is the size of the input. This efficiency makes the FFT an invaluable tool in various fields, including radar signal processing. It has been frequently used for doppler analysis transforming a set of M radar signals $s_n \in \mathbb{C}^K$ from a single receiver antenna n , where M is the number of radar frames in the time domain, to a Range-Doppler $S_{doppler}$ map in the frequency domain, which can be used to extract doppler-shifts caused by human vital signs [20][21][17]. FFT can be also used to extract Angle of Arrival information from the phase difference of ULA of N_{RX} receivers, transforming it into the frequency domain[6].

The N_{fft} -point FFT algorithm, where N_{fft} is the number of point used in the FFT calculation, takes in input the set of signals S from the N_{RX} receivers and the parameter N_{fft} that in our case is the number of considered angles N_a , and output a Range-Angle map S_{angle} .

2.1.5 MIMO UWB Radar

One of the most promising developments in UWB radar technology is the integration of Multiple-Input Multiple-Output (MIMO) systems. MIMO UWB radars can exploit the spatial diversity offered by ULAs to enhance detection capabilities. By arranging multiple antennas in a ULA configuration, these radars can achieve higher resolution and better target localization. The spatial diversity inherent in MIMO systems allows for more accurate estimation of the direction of arrival (DOA) of signals, in multiple dimensions, which is crucial for tracking and identifying targets in cluttered environments [25][16][1]. In fact, deploying physical antennas forming multiple ULAs of receivers disposed on orthogonal straight lines, it is possible to estimate both Azimuth and Elevation angles, respect to the device position and orientation.

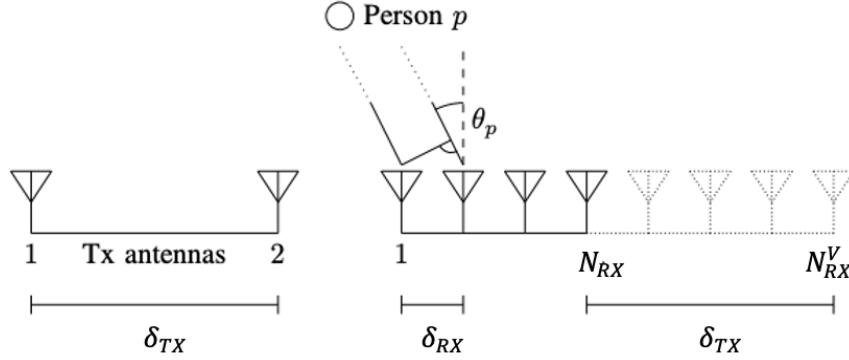


Figure 2: MIMO setup with $N_{TX}=2$ transmitters spaced of δ_{TX} , and $N_{RX} = 4$ receivers spaced of δ_{RX} , forming a vULA of N_{RX}^V receivers [25].

Since the distance between two adjacent antennas is far less than the distance between the human target and the receiver, it can be said that the reflected signals for each transmitting channel travel in parallel to each other and it is possible to increase the angular resolution by increasing the number of receivers in the ULA without deploying more physical antennas, using the phase differences at the different locations of the transmitter and receiver [1][25]. In particular, a ULA of N_{RX} receivers, equally spaced of a distance δ_{RX} , can be extended to a virtual ULA (vULA) of $N_{RX}^V = N_{TX} \times N_{RX}$ antennas, where N_{TX} is the number of antennas in the ULA of transmitters with a spacing of $\delta_{TX} = \delta_{RX} \times N_{RX}$, an example can be seen in Figure 2.

The imaging performance of UWB MIMO radar is related not only to the array configuration and number of elements, but also to the form of transmitted signals and application scenes [11].

Anyway, more antennas means more energy consumption and we will evaluate performance of 2D UWB radar imaging with different antennas configurations.

2.2. TinyML

TinyML, or Tiny Machine Learning, is an emerging field that focuses on developing machine learning models and algorithms that can run on low-power, resource-constrained devices such as microcontrollers. These devices are often part of the Internet of Things (IoT) and are used in applications where power consumption and size are critical factors.

This technology promises to revolutionize the way we implement smart applications by enabling machine learning capabilities on the very edge of the network, directly on the devices that are closest to the source of data.

This approach offers several key advantages, as highlighted by [3] including:

- **Energy Efficiency:** TinyML devices, due to their ultra-low power consumption, can operate efficiently, often on battery power for extended periods. This efficiency is crucial for applications where frequent battery replacement is impractical or where wired power is unavailable;
- **Privacy and Responsiveness:** By processing data locally on the device, TinyML can enhance privacy by reducing the need to send sensitive information to the cloud for processing. Local processing also minimizes latency, leading to more responsive applications;
- **Autonomy of Edge Devices:** TinyML enables edge devices to make decisions and take actions independently, without relying on continuous cloud connectivity. This autonomy is particularly important for applications in remote locations or where real-time decision-making is critical.

Previous works in the field have focused on various aspects of TinyML, including efficient neural network architectures, model compression techniques, and the deployment of deep learning models on ultra-low power microcontrollers. For instance, a comprehensive review by [14] discusses advancements in efficient neural networks for TinyML applications.

The development chain of TinyML applications involves several steps, from hardware setup and software setup to data collection, TinyML model training, firmware development, firmware compilation, and finally, TinyML operation. This process highlights the challenges of designing and deploying TinyML solutions, given the hardware constraints of tiny devices[24].

Main TinyML techniques comprises:

- **Model Compression:** involves reducing the size of a machine learning model without significantly affecting its performance. It's crucial for fitting complex models into devices with limited memory and computational power.
- **Quantization:** reduces the precision of the numbers used in a model's computations, which can decrease the model size and speed up inference while maintaining acceptable accuracy levels.
- **Neural Architecture Search:** automates the design of neural network architectures that are tailored to the constraints of TinyML devices, balancing performance and resource usage.

One of the most recent advances in TinyML is the execution of object detection algorithms on resource constrained devices. Since they are computationally very expensive and require a huge amount of annotated training data, even models designed for edge applications are hard to run on small devices. TinyML act to compress deep learning models to small dimensions whilst keeping its detection power through careful balancing among accuracy, speed, and memory footprint.

2.3. Object Detection

Object detection plays a pivotal role in the field of computer vision, facilitating the identification of objects within images or videos by determining their spatial location and boundaries and by categorizing these objects into distinct classes. Let X be an image, or a video frame, and let $\Omega = \{(y_1, b_1), \dots, (y_L, b_L)\}$ be a set of annotations, also called Ground Truth, where (y_l, b_l) represents the class and the position of the l -th object within the image. The position b is composed of 4 values, $(x_{min}, x_{max}, y_{min}, y_{max})$, which represents the portion of image in which the object can be found, called Bounding Box.

Taking X as input, an Object Detector must be able to generate a set $\Omega^{pred} = \{(y_1^{pred}, b_1^{pred}), \dots, (y_L^{pred}, b_L^{pred})\}$ of predictions where (y_l^{pred}, b_l^{pred}) represents the class and the Bounding Box of the l -th predicted object.

The quality of the predictions is evaluated by a scoring function $f(\Omega^{pred}, \Omega) = \{\sigma_1, \dots, \sigma_L\}$ where σ_l is the score of the l -th predicted object.



Figure 3: Example of real-time human detection in manufacturing production lines [4].

Traditional object detection methods comprise, in general, 3 main phases[13]:

- **Region Selection** is an essential step to identify the region of an object since the objects have different magnitude and aspect ratio and may occur at distinct regions of an image. Traditional approaches are based on the inspection of the entire image using multi-scale sliding window approach to detect objects. However this approach has high computational cost and also causes a large number of non-essential choices.
- **Features Extraction** algorithms are used to extract features from selected regions to provide a meaningful representation for object recognition. However, it is extremely hard to manually build a comprehensive feature descriptor that correctly identifies all kinds of object, because of contrasting backgrounds, lighting environment and perspective variances.
- **Classification** is used to identify the target objects and to build the models more organized and meaningful for visual perception.

2.3.1 Deep Learning Object Detectors

Deep learning-based object detection has significantly advanced the field of computer vision by enabling machines to identify and locate objects within images or videos with remarkable accuracy. With deep learning object detection networks and GPUs computing power, object detectors are incredibly flexible and can be trained for a wide range of tasks and custom applications such people counting, pedestrian recognition, face detection etc. This progress is largely attributed to the development and refinement of Convolutional Neural Networks (CNNs) to process and extract hierarchical feature representations from visual data, leveraging large amounts of labeled data (images annotated with bounding boxes and class labels) to learn patterns that distinguish different objects [13][27]. There are 2 main types of object detectors, categorized by the way they select areas of the input image which potentially contain objects, i.e. the *Region Proposal* algorithm [13]:

- **Two-Stage Object Detectors:** As the name suggest, they are characterize by 2 main steps, the first stage generates a set of *Region Proposal*—areas of the image that might contain objects. The second stage then classifies these regions into specific object categories and refines their bounding boxes. These models are known for their high accuracy and precision, better at detecting small objects, due to their systematic approach to scanning the image through the region proposal mechanism. Anyway, the 2-step process leads to a slower inference speed and more computationally intensive load. Most relevant Two-stage object detectors are **R-CNN**, **Fast R-CNN**, and **Faster R-CNN** [27].
- **One-Stage Object Detectors:** on the other hand, simplify the process by removing the *Region Proposal* step. Instead, they directly predict the class and bounding box for each location in the image in a single pass. This approach tends to be faster and less computationally expensive, making one-stage detectors more suitable for real-time applications. Generally, they achieve lower accuracy compared to Two-stage detectors depending by the analyzed environment. **YOLO** (You Only Look Once) and **SSD** (Single Shot Detector) are well known one-stage detectors[27].

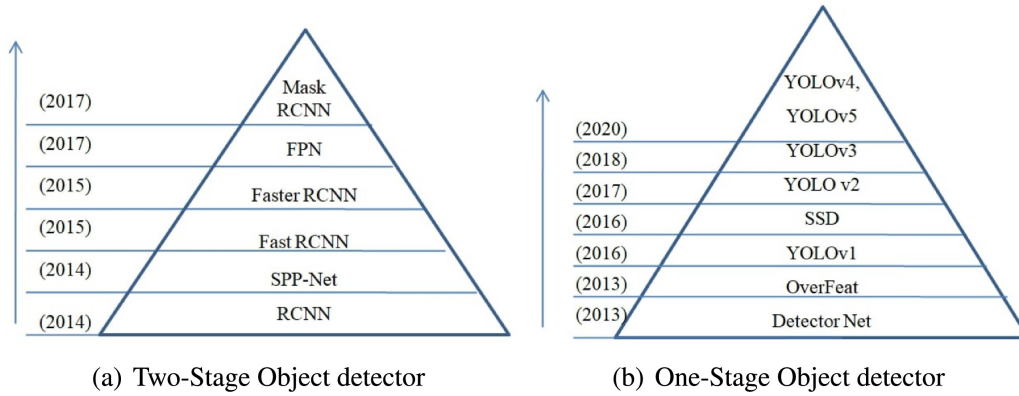


Figure 4: Example of Two-stage (a) and One-stage (b) object detectors developed in the years [13].

2.3.2 Faster-Objects More-Objects

Faster Objects, More Objects (FOMO) is a novel lightweight deep learning object detection model provided by Edge Impulse [9]. It was executed on a Cortex-M7 Micro Controller Unit, using 245KB of RAM and running object detection at 30 frames-per-second (fps)[9].

The efficiency of FOMO takes advantage from low computational power required by simple image classification adding localization and object count information, through a simplified version of object detection that is suitable for many use cases where the position of the objects in the image is needed but when a large or complex model cannot be used due to resource constraints on the device.

Differently from standard object detection algorithms, taking an input image X , FOMO is able to generate a set $\Omega^{pred} = \{(y_1^{pred}, C_1^{pred}), \dots, (y_L^{pred}, C_L^{pred})\}$ of predictions where (y_l^{pred}, C_l^{pred}) represents the class and the Centroid, i.e. the Bounding Box center of the l -th predicted object, and L is the number of objects. It exploits Transfer Learning, using early layers of *MobileNetV2* Features Extractor (FE) pre-trained on Image Classification tasks, to extract diffused lower-resolution image of the input, preserving its locality, that is then processed through N_{Conv} 2D convolutional layers of N_{Filt} 1×1 filters.

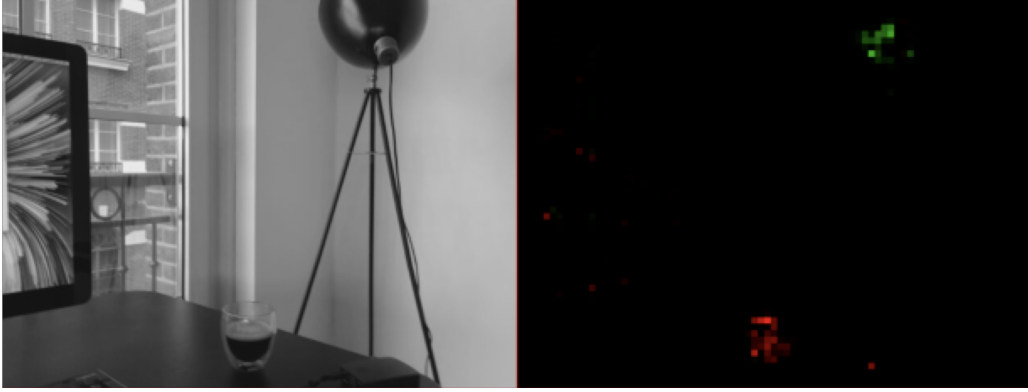


Figure 5: Example of probability map produced by FOMO given the input image on the left (for Cup object class in Red, Lamp object class in Green and Background in Black)[9].

Let $N_{Class}+1$ be the number of detected object classes plus background, then the final 2D convolutional layer of FOMO produces $N_{Class}+1$ per-region probability maps of size (H, W) , where a cell $m_n[i, j]$ of the n -th map represents the probability that the corresponding position in the input image X belongs to the n -th object class, such that:

$$\sum_{n=1}^{N_{Class}+1} m_n[i, j] = 1 \quad (5)$$

$\forall i \in \{1, \dots, H\}, \forall j \in \{1, \dots, W\}$.

Then highlighted areas in the n -th map, i.e. adjacent cells with probability $m_n[i, j]$ higher than a confidence threshold τ_c , represents areas in which the objects belonging to the n -th class can be found and from them is computed the (i,j) position of their centroids. An example of per-region probability map can be seen in Figure 5.

MobileNetV2 FE can be configured with different hyper-parameters such α that controls the number of filters inside layers of the FE, and the cut-point P_C , that is the output layer of the FE which controls the resolution (H, W) of the probability maps, setting also a trade-off between memory size and object size detection capability of the model.

FOMO is designed for camera-based object detection on constrained devices leveraging on a very lightweight architecture and post training quantization. In addition, training it on centroids of objects, makes it suitable for applications where it is not necessary to know the size of the object but its position is enough, like Figure 6.



Figure 6: Example of bees camera-based detection with FOMO, Red dots are detected bees' centroids[9].

Limitations In general FOMO works better at detecting objects of similar size since FOMO can be thought of as object detection where the bounding boxes are all square and, with the default $P_C = \text{block}_6$ configuration, 1/8th of the resolution of the input.

Moreover, objects shouldn't be too close to each other since during classification, the image splits into a grid and each cell is classified, then labels are collapsed to centroids, and two close objects may be collapsed to one. Adjacent boxes are then fused to the resulting bounding boxes. A classification is considered as correct if the predicted centroid is close enough to the centroid of the labeled box. It's thus not possible to detect distinct objects where their centroids occupy the same cell in the output. Although this can be optimized when increasing the image input resolution, as in Figure 7, or by choosing a different cutpoint P_C for the *MobileNetV2* FE leading to different detection granularity.

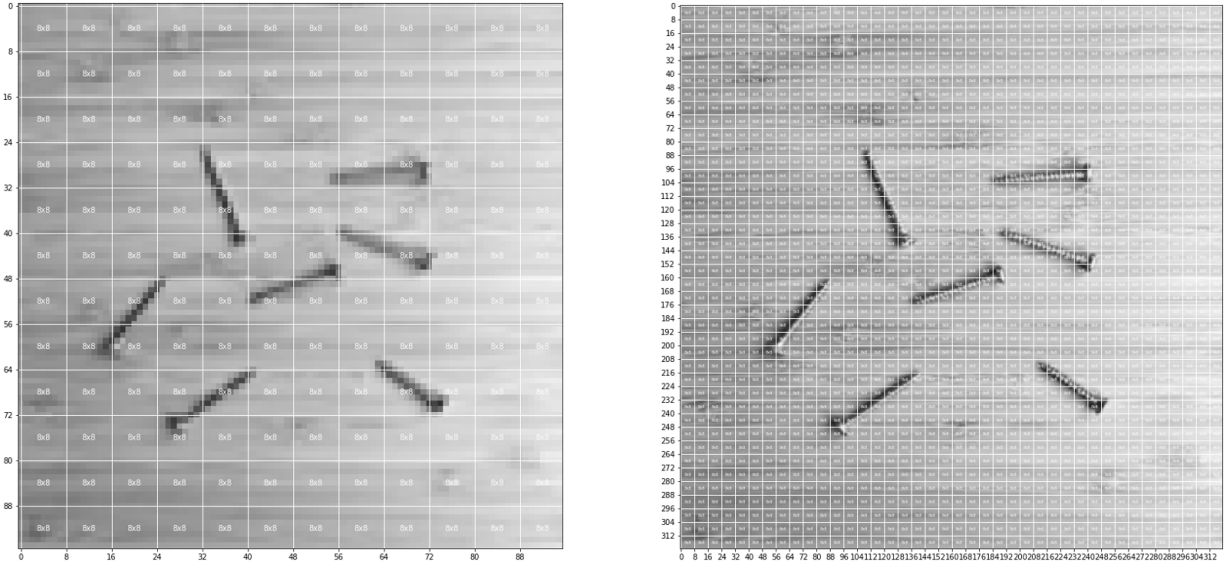


Figure 7: Example of FOMO image splitting into grid of 8×8 pixels cells, for images of shape $(96, 96)$ into a 12×12 grid (Left), and $(320, 320)$ into a 40×40 grid (Right)[9].

2.3.3 Performance Metrics

The goal of object detectors is to predict the location of objects of a given class in an image or video with a high confidence. As we formalize earlier, a detection can be represented by the class label y_l^{pred} and the bounding box b_l^{pred} of the detected object. The object detector associate to each prediction a confidence score σ_l , usually a value between 0 and 1 showing how confident the detector is about that prediction. Then we can evaluate the performance of the object detector on these 3 attributes with different metrics.

Precision and Recall Precision P is the ability of a model to identify only relevant objects. It is the percentage of correct positive predictions.

Recall R is the ability of a model to find all relevant cases (all ground-truth bounding boxes b_l). It is the percentage of correct positive predictions among all given ground truths [19].

To calculate the precision and recall values, each detected bounding box b_l must first be classified as:

- **True positive (TP)**: A correct detection of a ground-truth bounding box;
- **False positive (FP)**: An incorrect detection of a non-existing object or a misplaced detection of an existing object;
- **False negative (FN)**: An undetected ground-truth bounding box.

Assuming an image X with L ground-truths (y_l, b_l) and a model that outputs L^{pred} detections, of which L_{TP}^{pred} are correct ($L_{TP}^{pred} \leq L$), then P and R can be formally expressed as:

$$P = \frac{L_{TP}^{pred}}{L^{pred}} \quad (6)$$

$$R = \frac{L_{TP}^{pred}}{L} \quad (7)$$

The confidence score σ_l can be taken into account in the calculation of P and R by considering as positive detections only those whose confidence σ is larger than a confidence threshold τ , negative otherwise.

F1-Score The F1-score is defined as the harmonic mean of the precision P , calculated in (6), and recall R , calculated in (7), of an object detector, that is:

$$F_1 = 2 \times \frac{P \cdot R}{P + R} \quad (8)$$

It ranges from 0 to 1, where a score of 1 represents perfect precision and recall, and a score of 0 indicates the worst possible performance. The F1-score is particularly useful when you want to seek a balance between precision and recall, especially when the class distribution is unbalanced. As the F1-score does not take into account different confidence values, it is only used to compare object detectors in a fixed confidence threshold level τ [19].

3. Related Works

One of the most relevant research area in UWB technologies is the human localization in sectors like automotive and indoor environments, as also highlighted by [5].

For example, authors of [16] proposes a signal processing algorithm using a millimeter-wave Single-Input Multiple-Output (SIMO) Frequency Modulated Continuous Wave (FMCW) radar to automatically determine the number and 2D positions (azimuth angle and range) of multiple stationary human subjects.

It aims to isolate the Doppler signals caused by each subject's vital signs (cardiopulmonary activity) from static clutter and multipath reflections using Fast-Fourier Transform and Digital Beamforming to obtain Angle of Arrival of signals. The number of subjects is determined by counting the number of extracted vital signs signatures.

2D localization is performed by measuring the distance to the subjects' chests where the vital signs are sensed. [25] presents a complete signal processing pipeline for multi-person detection, localization and vital sign estimation using a stepped-frequency continuous wave (SFCW) UWB radar in degraded visibility conditions. Key steps includes clutter removal, virtual antenna array synthesis, 2D MUSIC localization algorithm, model order estimation for number of persons, and spatial filtering to extract vital signs like breathing rate. These works achieve state-of-art results in human localization, exploiting complex signal processing techniques which do not take into account resource limitations for on device localization tasks. There are few works which combine machine learning to UWB radars technology, for example authors of [26] designed *RODnet*, a deep learning object detection network using camera-radar fusion strategy to label the radar images from a FMCW radar and train their network to perform 3D localization in automotive through radar images. Despite it is a very powerful localization system, it does not take into account the heavy resource load of their deep learning model that cannot be executed on tiny devices.

Even less are works on TinyML and UWB radars, an example is [21] in which is introduced a novel TinyML solution for presence detection using Ultrawideband (UWB) radar data through a lightweight preprocessing pipeline, and a family of tiny convolutional neural networks (TyCNNs). [22], instead proposes using UWB radar and TyCNNs to develop privacy-preserving person counting and object detection solutions for tiny devices.

Nevertheless, no existing solution explores the usage of tinyML in 2D human indoor localization, even less using MIMO UWB radar imaging. Previous works, in fact, requires the collection of long windows of data while the proposed solution in this thesis works with instantaneous.

4. Problem Setting & Proposed Solution

The main purpose of this thesis is to develop a pipeline for 2D (Range-Azimuth) indoor human localization through UWB radar signals and evaluate its memory and computation efficiency for a constrained device.

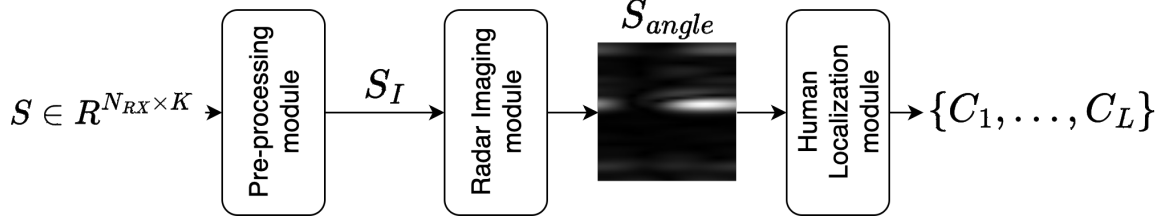


Figure 8: Indoor human 2D-Localization Pipeline

Using the same formalization introduced in *Section 2.1*, the pipeline must take in input the set S of radar acquisitions from N_{RX}^V antennas of a vULA, and output the number L and the Range-Angle(Azimuth) positions C_l of Human subjects within the scanned environment. The proposed solution is composed of 3 sub-modules:

- a *pre-processing module*;
- a *MIMO UWB radar imaging module* to map the set S to a Range-Angle map S_{angle} , characterized by a Range profile K and a vector of Angles Θ ;
- the *Human Localization on Radar Maps (HLRM) module*, to obtain from S_{angle} the number L and the positions $C_l = (r_l, \theta_l)$ of human targets, where (r_l, θ_l) are the Range and Azimuth coordinates within the input Range-Angle map.

The proposed Indoor human 2D-Localization Pipeline is depicted in Figure 8.

4.1. Pre-processing Module

The radar data S from N_{RX}^V antennas, are raw complex signals containing noise from static clutter reflections and a strong clutter interference in the early range bins given by the internal components of the UWB device, so we have to "clean" the signal with a low-pass filter, which preserve human targets' vital signs, before they can be processed by the MIMO radar imaging module.

For this reason, the *pre-processing module* exploits a moving average filter, as described in [25], and outputs the cleaned S_I . The clutter components are stationary along M radar frames in the acquisition time-domain, in contrast to human vital signs. Let $s_{n,m} \in S_m$ be the signal received by the n_{RX} -th element of the array, at the m -th radar frame, then we apply a sample moving average filter on each $s_{n,m} \in S_m$ separately, in the following way:

$$\tilde{s}_{n,m}[k] = s_{n,m}[k] - \frac{1}{W} \sum_{i=m-W+1}^m s_{n,i}[k] \quad (9)$$

for $m \in \{W, \dots, M\}$, where $W \leq M$ is the window dimension, i.e. the number of subsequent radar frames to be considered for clutter rejection, and $\tilde{s}_{n,m} \in S_{I,m}$ is the clutter-free signal from the n -th antenna at the m -th frame. Since the algorithm works on the time-domain, the first scan of each window must be rejected such that the algorithm needs to converge and it still contains the clutters.

4.2. MIMO UWB Radar Imaging Module

Then the *MIMO UWB radar imaging module*, implemented with DBF, takes in input the set of pre-processed radar acquisition $S_I \in \mathbb{C}^{N_{RX}^V \times K}$ from N_{RX}^V antennas and map it to a Range-Angle map $S_{angle} \in \mathbb{R}^{N_a \times K}$ using the DBF weights $W(\theta_i)$ of formula (4) in the following:

$$S_{angle}[i, :] = S[:, :] \times W(\theta_i) \quad (10)$$

$\forall i \in \{1, \dots, N_a\}$.

Angle-FFT We also try the visualization capabilities of the N_{fft} -point angle-FFT, performed on the set of pre-processed radar acquisition $S_I \in \mathbb{C}^{N_{RX}^V \times K}$ from N_{RX}^V antennas, where $N_{fft} = N_a$ is the number of angle points in the fft result we want to evaluate, as discussed in *Section 2.1.4*. The result is a Range-Angle map $S_{angle} \in \mathbb{R}^{N_a \times K}$ with slightly lower visual resolution and less angle resolution than the DBF algorithm, as depicted in Figure 9.

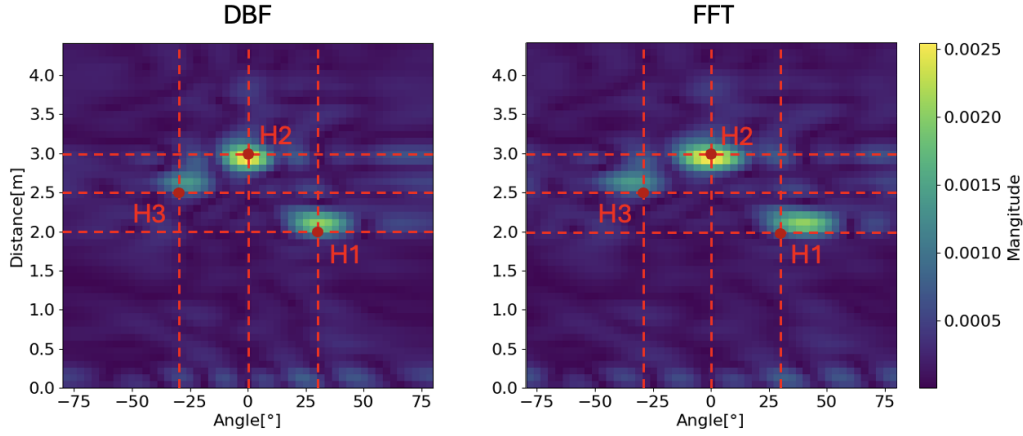


Figure 9: Visual example of Range-Angle radar maps obtained with DBF(*Left*) and angle-FFT(*Right*) with 3 Human Targets at distance and angle $(2m, +30^\circ)$, $(2.5m, -30^\circ)$ and $(3m, 0^\circ)$

The reason is that the angle-FFT is an implementation of the DBF algorithm, widely used to optimize beam calculation of more complex ULAs with a huge number of N_{RX} antennas [15]. Moreover, the possible optimization given by the FFT is strongly dependent by the algorithm used to implement it on device, then we decided to implement the *MIMO UWB radar imaging module* using the Digital Beamforming formalization, since it better represents the problem faced in this work and we are considering low power consuming devices with few N_{RX} antennas which does not impact the performance of the Range-Angle map generation algorithm.

Normalization and Rescaling The values of a Range-Angle radar map S_{angle} computed by the *MIMO UWB radar imaging module* need to be normalized in the range $[0; 1]$ before it can be processed by the *HLRM module*. This process ensures that all values in S_{angle} are scaled to a uniform range, which can be particularly beneficial for improving the performance and convergence speed of machine learning models. This can be achieved through the following formula:

$$S_{angle}[i, j] = \frac{S_{angle}[i, j] - \min(S_{angle})}{\max(S_{angle}) - \min(S_{angle})} \quad (11)$$

$\forall i \in \{1, \dots, N_a\}, \forall j \in \{1, \dots, K\}$.

The S_{angle} radar map is then rescaled to $N_a = N_{FE}$ and $K = K_{FE}$, where (N_{FE}, K_{FE}) is the input shape of the *Feature Extractor* used for the *HLRM module*.

4.3. Human Localization on Radar Maps Module

The *HLRM module* adapts the FOMO [9] algorithm to work on radar maps, instead of the images it was designed to work on. The algorithm is well fit for the task, since Human targets are just a kind of "globe" within UWB radar Range-Angle images, and we just need to localize and count them. The formalization is very similar to the one presented in *Section 2.3* with the difference that in our solution the input is a Range-Angle radar map S_{angle} , containing $\Omega = \{(y_1, b_1), \dots, (y_L, b_L)\}$ human targets, and the output is the set $\Omega^{pred} = \{(y_1^{pred}, C_1^{pred}), \dots, (y_L^{pred}, C_L^{pred})\}$ of L detected humans and their centroid positions $C_l = (r_l, \theta_l)$. In particular, we want to detect only objects of class *Human*. Consequently, the number of probability maps produced by the *HLRM module* last convolutional layer is $N_{Class} + 1 = 2$, considering also the *Background* class.

The *HLRM module* architecture relies on the following parameters:

- the *object_weight* w_{obj} parameter can be used to balance situations in which the objects to detect are very few compared to the background;
- α controls the number of filters inside layers of the FE;
- cut-point P_C , that is the output layer of the FE;
- N_{Conv} 2D convolutional layers before the output one;
- and the number of filters N_{Filt} inside them.

The algorithm is trained with an optimizer Opt and a learning rate l_r .

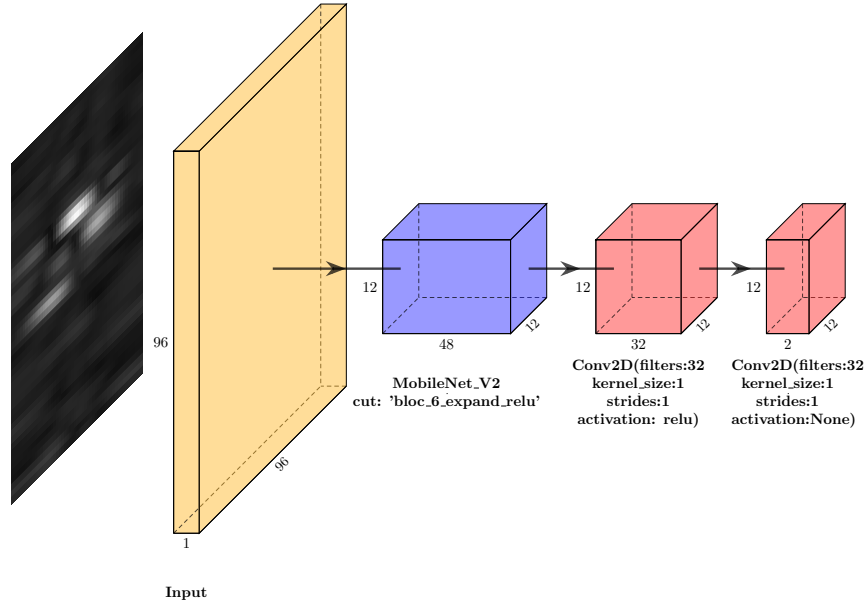


Figure 10: *HLRM module* baseline architecture with $P_C = block_6$, $\alpha = 0.35$, $N_{Conv} = 1$, $N_{filt}=32$ and $w_{obj} = 100$.

The *HLRM module* has to be runnable on constrained devices, so it has to respect memory and execution limits given by the device used. Therefore, this research aims to evaluate the detection capabilities of the proposed pipeline with Range-Angle maps generated from different antenna configurations, considering that each physically deployed antenna leads to more energy consumption. *Section 5* contains experiments for different UWB devices with different number of antennas used. Each configuration is used to train and to test the *HLRM module* with the hyper-parameters previously described to find the best ones that maximize its performance in human indoor 2D-Localization, starting from the Baseline model in Figure 10.

5. Experimental setting

This section provide an overview on the experiments conducted to evaluate our proposed solution, using the formalization in *Section 2*.

To train and then test the performance of our Pipeline, we used 2 datasets composed of radar scans of an indoor environment, setting 3 vULA configurations for our experiments:

- **8-RX** and **4-RX** vULAs with Multi-Person Localization and Vital Sign Estimation Radar Dataset [25] collected with Vayyar's SFCW MIMO UWB radar, *Section 5.1* ;
- **3-RX** vULA dataset collected with Novelda X7 MIMO UWB Radar, *Section 5.2*.

Each dataset is composed by a set of M radar scans $S \in \mathbb{C}^{N_{RX} \times K}$ for each N_{TX} of the MIMO radar. Then *MIMO UWB radar imaging module* have been used to obtain a dataset of M Range-Angle maps $S_{angle} \in \mathbb{R}^{N_a \times K}$ for the relative antennas configuration.

Each scenario is splitted into 80% for training and 20% for testing purposes, and values of each S_{angle} is normalized in the range $[0; 1]$, before the model training phase.

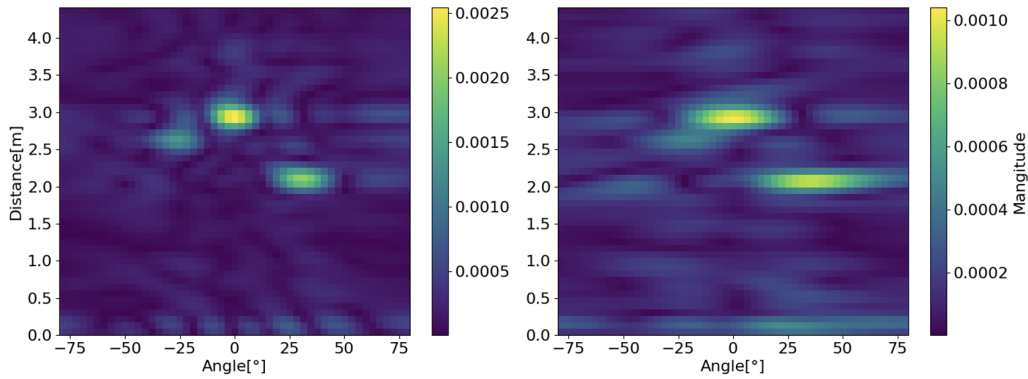


Figure 11: Range-Angle radar maps of **8RX**(Left) and **4RX**(Right) vULAs, with 3 Human Targets at distance and angle $(2m, +30^\circ)$, $(2.5m, -30^\circ)$ and $(3m, 0^\circ)$

5.1. Multi-Person Localization Radar Dataset

This dataset is collected by authors of [25] with a SFCW MIMO UWB radar, by manufacturer Vayyar, for *through-wall vital signs estimation through UWB radar* purpose. The radar operates at central frequency f_c of 7.15GHz with a bandwidth B of 1.7GHz, and is composed by 18 antennas disposed in a grid with an inter-element spacing $\delta_{RX} = 2cm$, as shown in Figure 12, each one sampling a signal of $K = 137$ range bins, with a range resolution of 8.8cm. It contains 62 scenarios, sampled at a frequency $f_s = 10Hz$, with up to five static persons captured in different postures, angles and ranges including obstacles that block the radar line of sight. A subset of 14 scenarios for a total of $M = 13200$ radar scans, have been selected for our experiments, removing ones which comprise obstacles and human targets too close to be theoretically sensed by the UWB radar system as 2 separated targets. Using only scans from the 2TX and 4RX antennas highlighted in Figure 12, we are able to synthesize a vULA of $N_{RX}^V = 8$ and a vULA of $N_{RX}^V = 4$ to be processed by the *MIMO UWB radar imaging module*. Then we obtain a Range-Angle map dataset for each antenna configuration, with $M = 13200$, $N_a = 51$ angles in range $[-80^\circ; +80^\circ]$, and $K = 51$ range bins. A bounding box is generated around each target in each map using meta data information on target positions provided by [25].

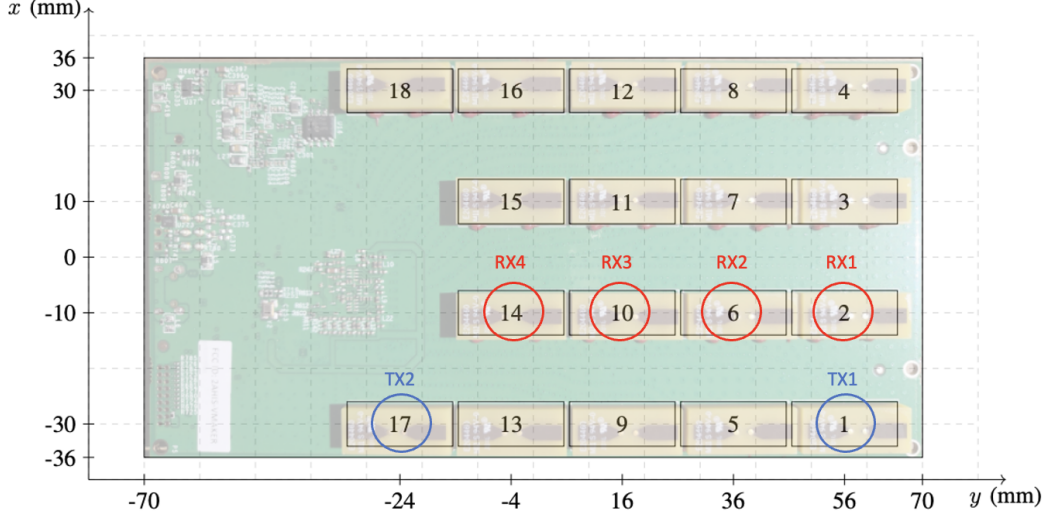


Figure 12: Walabot antennas disposition [25], highlighting TX(Blue) and RX(Red) antennas used to build the vULA used for this work.

5.2. Novelda X7 dataset

The X7, by manufacturer Novelda, is a MIMO UWB impulse radar provided of only 2 transceiver antennas, $N_{RX} = N_{TX} = 2$, designed for robust low power presence detection and beamforming applications, Figure 14. A transceiver antenna can act as both transmitter and receiver in a synchronized way, allowing to synthesize a vULA of $N_{RX}^V = 3$, since $\delta_{TX} = \delta_{RX}$ since, with inter-element spacing $\delta_{RX} = 2\text{cm}$. This radar operates at a center frequency $f_c = 7.875\text{GHz}$, a bandwidth $B = 750\text{MHz}$, and a range profile of $K = 192$ bins, with a range resolution of 7.14cm . We used it to collect radar scans from only 2 physically deployed antennas, of a simpler environment with a single static human target (y, C) , located in 9 different combinations of distances $\{1\text{m}, 2\text{m}, 3\text{m}\}$ and angles $\{-35^\circ, 0^\circ, +35^\circ\}$, for a total of $M = 9000$ radar scans. Then using the *MIMO UWB radar imaging module* we obtain the Range-Angle map dataset for the $N_{RX}^V = 3$ vULA antennas configuration, with $M = 9000$, $N_a = 51$ angles in $[-60^\circ; +60^\circ]$, and $K = 51$ range bins up to 3.6 meters. A bounding box b_1 is generated around the y_1 human target in each map using the relative experimental position.

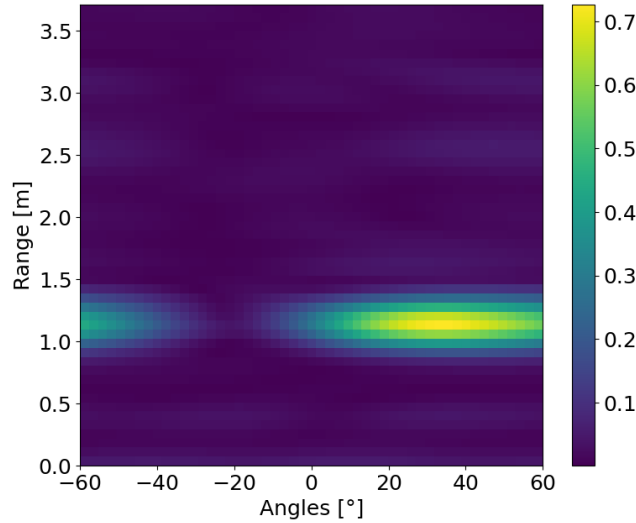


Figure 13: Range-Angle radar map from the **3RX** vULA, with a Human Target at distance and angle $(1\text{m}, +35^\circ)$

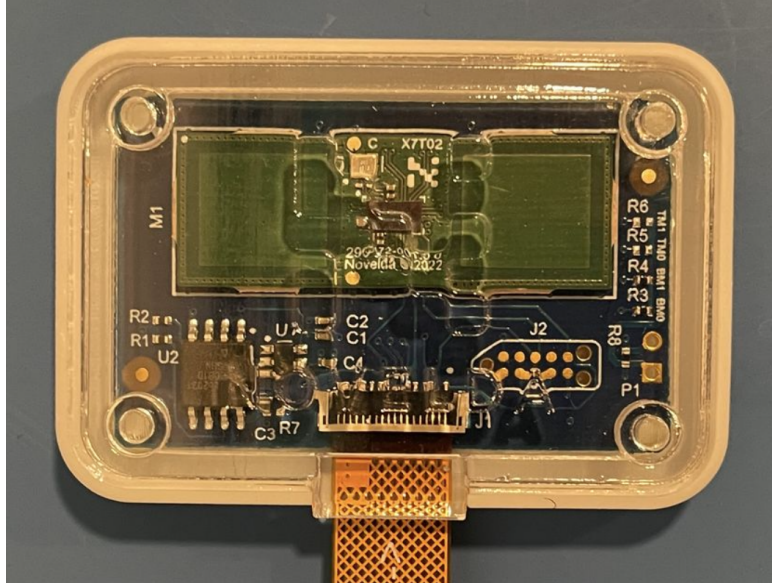


Figure 14: Novelda x7 MIMO UWB pulse radar

Data Logger The data collection process for the **3RX** setting, required to implement a *DataLogger* to communicate with X7 MIMO UWB Radar through the API (Application programming interface) provided by Novelda [18]. The *DataLogger* has been implemented in Python and it is used to set various device parameters to collect radar scans from the MIMO UWB radar, such as the sampling frequency $f_s = 10Hz$, which is sufficient to collect human vital signs which are in the range of $[0.8; 1.5]Hz$, or the number $N_{RX} = 2$ of receivers and $N_{TX} = 2$ of transmitters antennas we used.

The *X7 – API* allows to set a static clutter removal coefficient $c_{static} \in [0; 1]$ which controls the response of an on-device low-pass filter to remove static clutter noise from the UWB scans. A lower value will result in a faster response time, while a higher value will result in a slower response time of the filter, than we set it to $c_{static} = 0.8$ since we are analyzing a static environment with static human target, while faster response is suggested for motion detection purposes. Once the radar parameters are configured, the *DataLogger* runs the data collection for a single scenario, until $M = 1000$ radar scans have been collected. During the collection, a callback function *collect_scan()* is executed each time a radar scan S is received allowing us to process and visualize the radar data in real-time. When the data collection process is complete, the radar scans are stored in a *csv* format file, which allows to better organize matrices of data.

5.3. Experiments

The experiments were conducted to evaluate the performance of our Pipeline in **8-RX**, **4-RX** and **3-RX** vULAs settings, from respectively 6, 5 and 2 physically deployed antennas. For each setting, we evaluated visually the performance of the *MIMO UWB radar imaging module*. The results of imaging process can be seen in Figure 11. Furthermore, we evaluated the effects on the various hyperparameters of the *HLRM module* algorithm on a validation dataset (composed of 20% of the data of the training set that were excluded from the training). We used this validation process to select the best hyperparameters for this algorithm, for the 3 different settings.

We evaluated the validation *F1 – score* $\in [0; 1]$. This metric is useful in applications where both the number and the quality of accurate predictions are important, assessing the model’s ability to correctly identify objects while minimizing the number of false positives and false negatives. The training process is performed with EARLYSTOPPING to reduce overfitting, REDUCELRONPLATEAU and *Opt = Adam*. The validation of **8-RX** and **4-RX** settings comprised different combinations of the *HLRM module* hyperparameters listed in Table 1 .

Hyperparameter	Tested values
P_C	$\{block_3, block_6, block_9\}$
α	$\{0.1, 0.35\}$
w_{obj}	$\{50, 100, 500\}$
N_{Conv}	$\{1, 2\}$
N_{filt}	$\{32, 64, 128\}$
lr	$\{0.01, 0.001, 0.001\}$

Table 1: Tested *HLRM module* hyperparameters.

To evaluate the **3-RX** setting, we train the *HLRM module* with the hyper-parameters of the best 3 models for the **4-RX** setting. In Section 6 is resumed the performance of the best model for each antennas configuration setting, tested on the relative testing dataset. The performance of tested models is evaluated on *Accuracy* score, defined as the percentage of testing radar scans with Precision $P > 80\%$, where $P = \frac{\text{true positive detected humans}}{\text{all detected humans}}$ within a radar scan S .

The focus on Precision and setting a threshold highlights the importance of not just detecting humans, but detecting them accurately with minimal false positives. *Accuracy* is used to compare object detection performance of models that uses different metrics on predicted bounding boxes , like *YoloV5* [12].

Edge Impulse Studio Edge Impulse Studio is a platform that enables developers to create and deploy machine learning models on edge devices efficiently. It provides tools for building, training, and deploying EdgeML and TinyML models for various applications such as on-device Image-Classification, Face Recognition, Object Detection, Audio Classification etc.[9]. We used it to manage the training and the test phase of the models tried in each experimental settings previously presented. The *HLRM module* architecture has been configured through the *ExpertMode* of the Edge Impulse platform, which allows to build a machine learning model through python libraries such as *Keras*[7], *TensorFlow*[10], and *TensorFlowLite* [8] which is an open-source ML inference framework for running deep-learning models on embedded systems.

```
def build_model(input_shape: tuple, weights: str, alpha: float,
               num_classes: int) -> tf.keras.Model:

    mobile_net_v2 = MobileNetV2(input_shape=input_shape,
                                weights=weights,
                                alpha=alpha,
                                include_top=True)

    for layer in mobile_net_v2.layers:
        if type(layer) == BatchNormalization:
            layer.momentum = 0.9

    cut_point = mobile_net_v2.get_layer('block_6_expand_relu')
    model = Conv2D(filters=32, kernel_size=1, strides=1,
                  activation='relu', name='head')(cut_point.output)
    logits = Conv2D(filters=num_classes, kernel_size=1, strides=1,
                  activation=None, name='logits')(model)
    return Model(inputs=mobile_net_v2.input, outputs=logits)
```

Figure 15: Python function to build the *HLRM module* model.

An example of a snippet of code used in this work is depicted in Figure 15. For each experimental setting, the relative training and test datasets of range-angle radar maps have been uploaded on the Edge Impulse Studio platform, that allows also to normalize and rescale them to the input shape of the machine learning model to be trained. In our case a Range-Angle radar map S_{Angle} of shape $K = 51$ and $N_a = 51$ is upscaled to the shape $K = 96$ and $N_a = 96$ to match the input shape of the FOMO *Features Extractor*, since it has been pre-trained on images of shape (96,96) and using the same shape can enhance the *Transfer Learning* process on our models. On this platform, it is also possible to train machine learning models developed by the *Edge Impulse Community* engineers for EdgeML applications such as *YoloV5*. This modified version of *YoloV5* contains a learning rate scheduler to find the optimal l_r value and an *Autotune* feature for the optimal $batch_size$ before the training process starts. We trained it on our experimental datasets, evaluating the effects of different number of *Epochs* $\in \{10, 30, 60, 80, 120\}$ on a validation dataset (composed of 20% of the data of the training set that were excluded from the training).

Finally, the Edge Impulse Studio platform enables to profile the validation score of both unoptimized *float_32* and post-training quantized *int_8* version of trained models, using the F1-score $F1$ for *HLRM module*, and the Precision P for *YoloV5* as described in Section 5.3. Then we tested the *float_32* models comparing their performance based on the *Accuracy* score, with the limitation that Edge Impulse does not allows to test the optimized *int8* model until it is deployed on a compatible embedded device.

6. Results

Now we present the results of the experiments on our Human Indoor 2D-Localization Pipeline with different antennas configuration. The results of the validation process on the *HLRM module* hyperparameters can be seen in Table 2 for the **8RX** and **4RX** settings and in Table 3 for the **3RX** setting. The best hyperparameters for the 3 different settings results to be:

- $\{P_C = \text{block}_6, \alpha = 0.35, l_r = 0.01, w_{obj} = 50, N_{Conv} = 1, N_{filt} = 64\}$ for the **8RX**;
- $\{P_C = \text{block}_6, \alpha = 0.35, l_r = 0.01, w_{obj} = 50, N_{Conv} = 2, N_{filt} = 32\}$ for the **4RX**;
- $\{P_C = \text{block}_6, \alpha = 0.35, l_r = 0.001, w_{obj} = 100, N_{Conv} = 2, N_{filt} = 32\}$ for the **3RX**.

An EdgeML version of *YoloV5* provided as a *Community Learn Block by Edge Impulse*[23], having a model size of 3.4MB, has been trained and tested on same datasets of our experimental settings.

The results of the validation process on the *Yolov5* hyperparameters is depicted in Table 4 and the best validation performance is given by the model trained with *Epochs* = 60, *Epochs* = 120 and *Epochs* = 30 for the 3 different experimental settings. We evaluated also the inference time, the flash memory footprint and the peak of RAM usage of the *float_32* model versions on a virtual environment simulating a RaspberryPi4, of both *Yolov5* and the proposed solution.

They are reported with a comparison of test accuracy of best models for each experimental setting, in Table 5, that highlights a very good performance for the **8-RX** setting, which degrades in the **4-RX** as we expected since with less antennas the UWB sensing capability is reduced resulting in more noisy Range-Angle maps and then a reduced human localization performance.

Moreover the *HLRM module* models outperforms *Yolov5* models in accuracy, memory and execution time, demonstrating to be a very lightweight, high performing, detection models also in a more constrained setting as the **3-RX** one, with only 2 physically deployed antennas, less bandwidth and a single target to detect. The large difference in results, may be also due to the relatively limited dimensions of the training datasets: it is possible that *YoloV5*, being a large architecture requires a larger amount of training data in order to be trained appropriately.

A visual example of the performance of our *HLRM module* is shown in Figure 16 and and Figure 17 in which is reported respectively the result of a good prediction and a bad prediction for each experimental settings, compared to their relative ground truth bounding boxes (*Left*). Figure 16 confirm the very high performance of our solution also with noisy Range-Angle radar maps, while its limitations are clearly highlighted in Figure 17 in which the module failed to detect human targets where the sensing capabilities of the UWB device used are not able to detect them.

8RX	P_C	α	N_{conv}	N_{filt}	w_{obj}	l_r	Val F1(float32)	Val F1(int8)
Baseline	$block_6$	0.35	1	32	100	0.001	86.1%	85.7%
P_C, N_{Conv}, N_{filt}	$block_6$	0.35	1	64	100	0.001	93%	92.5%
	$block_6$	0.35	2	32	100	0.001	91.3%	91.3%
	$block_3$	0.35	2	32	100	0.001	85.1%	84.1%
	$block_3$	0.35	1	64	100	0.001	84.7%	84.2%
	$block_3$	0.35	2	64	100	0.001	83.6%	83.5%
	$block_3$	0.35	1	128	100	0.001	84.5%	84.6%
	$block_9$	0.35	1	32	100	0.001	76.3%	76.2%
	$block_9$	0.35	2	16	100	0.001	76.1%	76.1%
	$block_9$	0.35	1	64	100	0.001	76.3%	76.4%
α	$block_6$	0.1	1	64	100	0.001	90.1%	91%
w_{obj}	$block_6$	0.35	1	64	500	0.001	87.1%	86.8%
	$block_6$	0.35	1	64	50	0.001	93.7%	93.8%
l_r	$block_6$	0.35	1	64	50	0.0001	92.9%	90.4%
	$block_6$	0.35	1	64	50	0.01	95%	94.8%
4RX	P_C	α	N_{conv}	N_{filt}	w_{obj}	l_r	Val F1(float32)	Val F1(int8)
Baseline	$block_6$	0.35	1	32	100	0.001	84.1%	84.7%
P_C, N_{Conv}, N_{filt}	$block_6$	0.35	1	64	100	0.001	83.9%	84.2%
	$block_6$	0.35	2	32	100	0.001	84.8%	85.4%
	$block_3$	0.35	2	32	100	0.001	72.7%	72.2%
	$block_3$	0.35	1	64	100	0.001	78.9%	78.1%
	$block_3$	0.35	2	64	100	0.001	73.6%	73.5%
	$block_3$	0.35	1	128	100	0.001	79.1%	78.4%
	$block_9$	0.35	1	32	100	0.001	75.9%	75.9%
	$block_9$	0.35	2	16	100	0.001	76.1%	76%
	$block_9$	0.35	1	64	100	0.001	75.9%	76%
α	$block_6$	0.1	2	32	100	0.001	88%	88.3%
w_{obj}	$block_6$	0.35	2	32	500	0.001	81.7%	82.8%
	$block_6$	0.35	2	32	50	0.001	90.4%	90.6%
l_r	$block_6$	0.35	1	64	50	0.0001	86.8%	86%
	$block_6$	0.35	1	64	50	0.01	93.4%	93.7%

Table 2: *HLRM* module hyperparameters tested for **8RX** and **4RX** settings. The validation F1-score has been reported for both *float32* and post training quantized *int8* model versions.

3RX	P_C	α	N_{conv}	N_{filt}	w_{obj}	l_r	Val F1(float32)	Val F1(int8)
	$block_6$	0.35	2	32	50	0.01	97.4%	96%
	$block_6$	0.1	2	32	100	0.001	97.1%	93.7%
	$block_6$	0.35	2	32	100	0.001	99.3%	98.1%

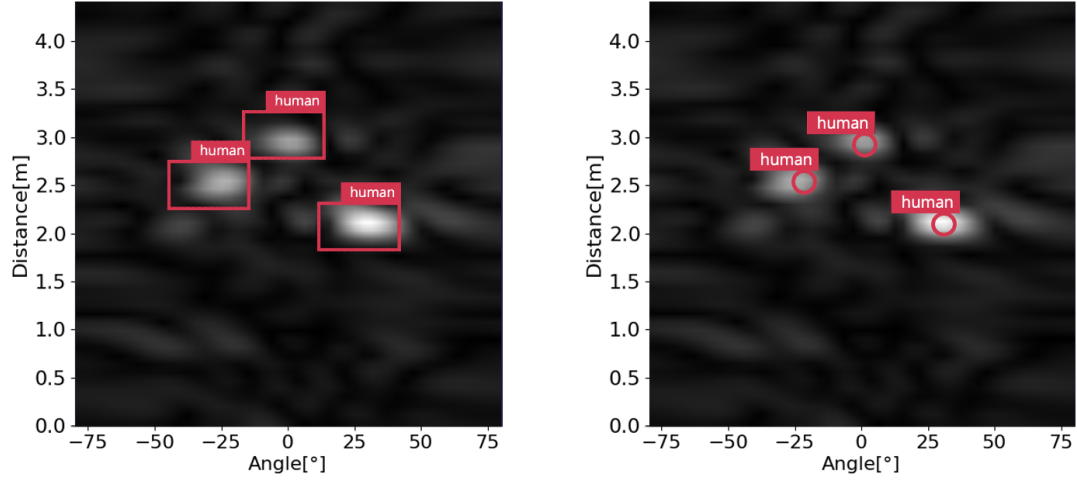
Table 3: *HLRM* module hyperparameters tested for the **3RX** setting. The validation F1-score has been reported for both *float32* and post training quantized *int8* model versions.

8RX	<i>Epochs</i>	<i>Batch_Size</i>	Val Precision(float32)	Val Precision(int8)
	30	16 (Autotune)	91.5%	87.3%
	80	16 (Autotune)	93.9%	92%
	60	16 (Autotune)	94.2%	89.3%
4RX	<i>Epochs</i>	<i>Batch_Size</i>	Val Precision(float32)	Val Precision(int8)
	60	16 (Autotune)	69.3%	64.7%
	80	16 (Autotune)	71.4%	67.8%
	120	16 (Autotune)	73.4%	68.7%
3RX	<i>Epochs</i>	<i>Batch_Size</i>	Val Precision(float32)	Val Precision(int8)
	10	16 (Autotune)	71.5%	69.3%
	30	16 (Autotune)	92.5%	90%

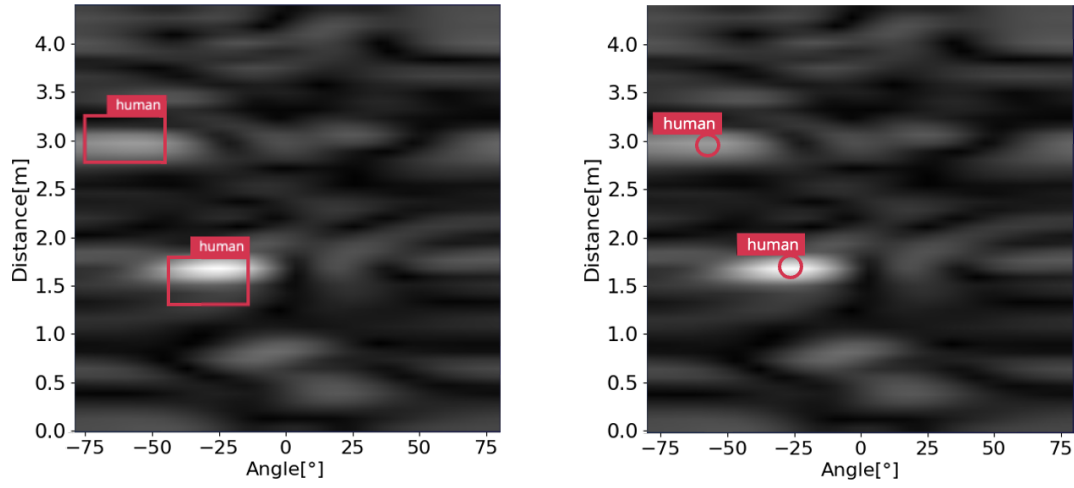
Table 4: *YOLOv5* hyperparameters tested for the **8RX**, **4RX** and **3RX** settings. The validation Precision score has been reported for both *float32* and post training quantized *int8* model versions.

8RX	Peak RAM	Flash Memory	Exec. Time	Test Acc.
HLRM	887.1KB	113.7KB	3ms	92.8%
YoloV5	6MB	3.4MB	26ms	85.5%
4RX	Peak RAM	Flash Memory	Exec. Time	Test Acc.
HLRM	887.3KB	105.5KB	3ms	83.8%
YoloV5	6MB	3.4MB	26ms	25.4%
3RX	Peak RAM	Flash Memory	Exec. Time	Test Acc.
HLRM	887.3KB	105.5KB	3ms	98.4%
YoloV5	6MB	3.4MB	26ms	83.3%

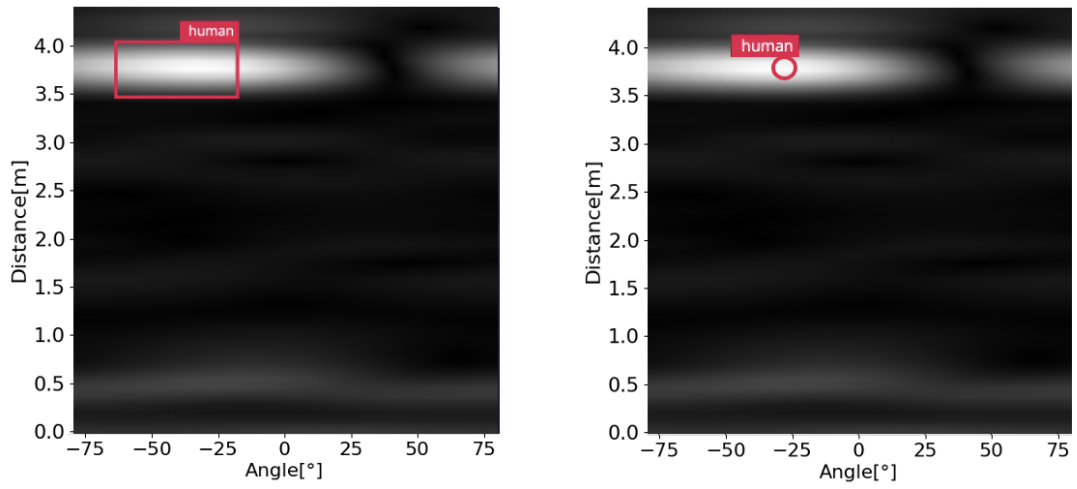
Table 5: Localization Test results on **8RX**, **4RX** and **3RX** experimental datasets. The *Peak RAM* and *Flash Memory* usage is reported for the *float32* versions of tested models.



(a) Example of good detection on 8RX dataset.

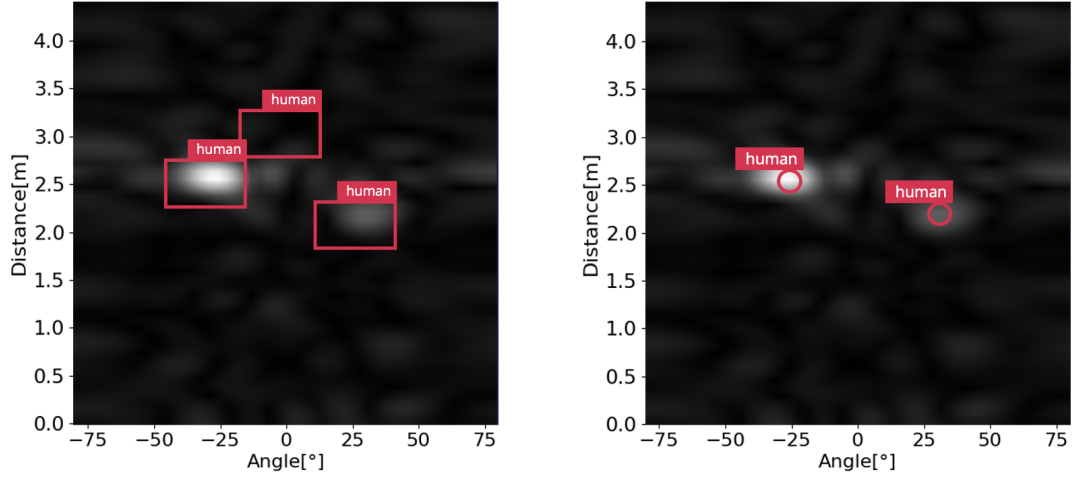


(b) Example of good detection on 4RX dataset.

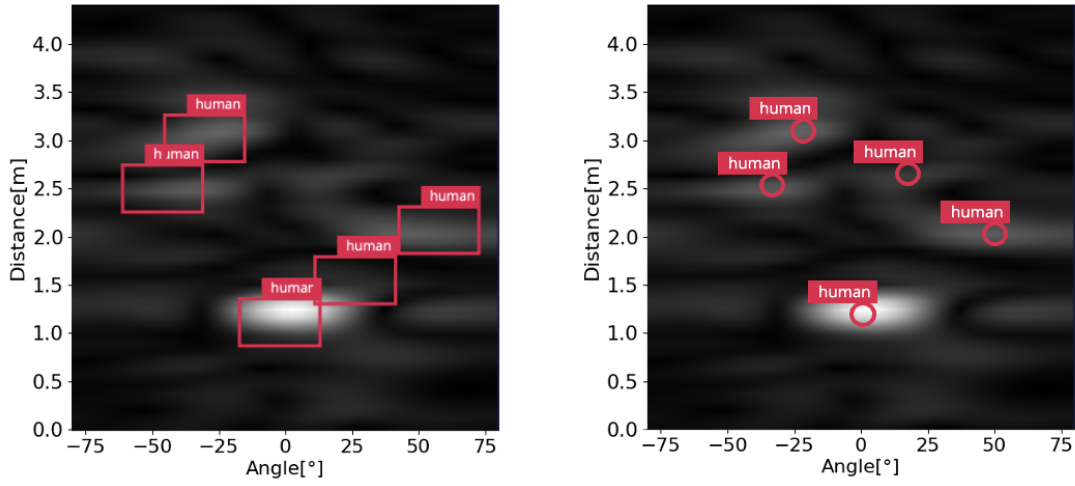


(c) Example of good detection on 3RX dataset.

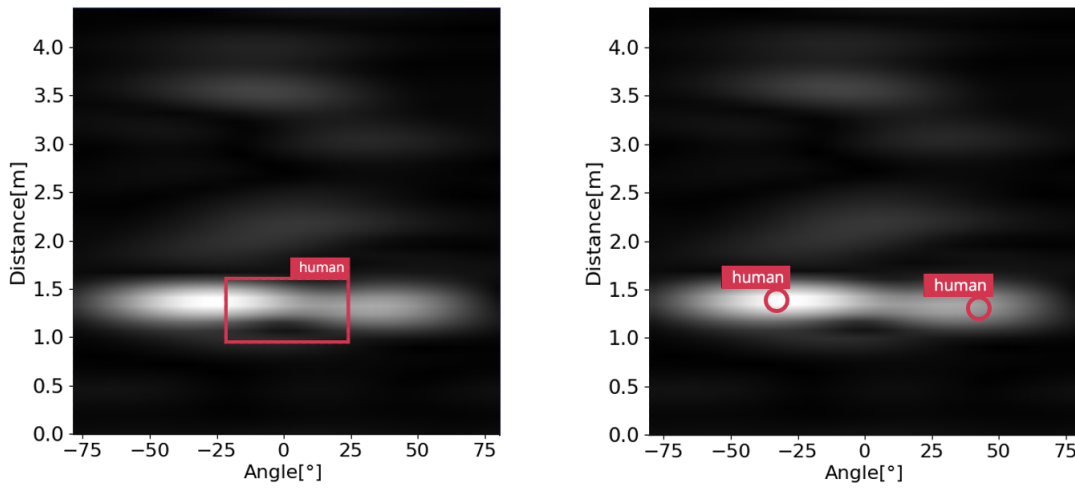
Figure 16



(a) Example of wrong detection on 8RX dataset.



(b) Example of wrong detection on 4RX dataset.



(c) Example of wrong detection on 3RX dataset.

Figure 17

7. Conclusions

This thesis demonstrates the effectiveness of combining UWB technology and TinyML algorithm for human indoor localization through radar maps in a controlled environment. We showed how combining different antennas configurations it is possible to obtain different localization performance, making our solution suitable for various applications depending on the desired *memory footprint/energy consumption* trade-off.

The proposed solution could be employed for human localization and counting in privacy-centric environments such as elderly patient monitoring, working environment monitoring or bathrooms occupancy monitoring.

Future directions include the porting of our solution on a tiny device to test its real-time capabilities on a physical device. The most challenging part of this work is to collect a dataset that is feasible for training our solution, due to the lack of research on the topic. It may then be useful to extend the dataset collected in this work with more complex scenarios to make the solution more robust in different environments, trying also to automate and enhance the dataset labelling process with camera-radar fusion techniques such as in [26].

Finally, UWB radar maps provided a valuable visual representation of the analyzed environment, then optimizing or modifying the TinyML architecture it could be possible to extend the human localization with tracking capability by extract relevant features from a stream of radar maps, or to obtain information other than the number and positions of targets, for task such as 2D gesture recognition.

References

- [1] Shahzad Ahmed, Junbyung Park, and Sung Ho Cho. Effects of receiver beamforming for vital sign measurements using fmcw radar at various distances and angles. *Sensors*, 22:6877, SEP 2022.
- [2] Yasir Ahmed. Fundamentals of a Uniform Linear Array (ULA), 2018. Accessed on March 03, 2024.
- [3] Colby R. Banbury, Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, Robert Hurtado, David Kanter, Anton Lokhmotov, David Patterson, Danilo Pau, Jae sun Seo, Jeff Sieracki, Urmish Thakker, Marian Verhelst, and Poonam Yadav. Benchmarking tinyml systems: Challenges and direction, 2021.
- [4] Gaudenz Boesch. Object Detection in 2024: The Definitive Guide by viso.ai, 2024.
- [5] Mohammad Cheraghinia, Adnan Shahid, Stijn Luchie, Gert-Jan Gordebeke, Olivier Caytan, Jaron Fontaine, Ben Van Herbruggen, Sam Lemey, and Eli De Poorter. A comprehensive overview on uwb radar: Applications, standards, signal processing techniques, datasets, radio chips, trends and future research directions, 2024.
- [6] Ushemadzoro Chipengo, Arien Sligar, and Shawn Carpenter. High fidelity physics simulation of 128 channel mimo sensor for 77ghz automotive radar. *IEEE Access*, 8:160643–160652, 2020.
- [7] François Chollet et al. Keras. <https://keras.io>, 2015.
- [8] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Shlomi Regev, Rocky Rhodes, Tiezhen Wang, and Pete Warden. Tensorflow lite micro: Embedded machine learning on tinyml systems, 2021.
- [9] Inc Edge Impulse. Fomo: Object detection for constrained devices, 2022.
- [10] Martín Abadi et al. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [11] Zhipeng Hu, Zhaofa Zeng, Kun Wang, Weike Feng, Jianmin Zhang, Qi Lu, and Xiaoqian Kang. Design and analysis of a uwb mimo radar system with miniaturized vivaldi antenna for through-wall imaging. *Remote Sensing*, 11(16), 2019.
- [12] Glenn Jocher and Sergiu Wadmann. Comprehensive Guide to Ultralytics YOLOv5, 2018. Accessed on February 15, 2024.
- [13] Ravpreet Kaur and Sarbjit Singh. A comprehensive review of object detection with deep learning. *Digital Signal Processing*, 132:103812, 2023.
- [14] Minh Tri Lê, Pierre Wolinski, and Julyan Arbel. Efficient neural networks for tiny machine learning: A comprehensive review, 2023.
- [15] Arjuna Madanayake, Renato J. Cintra, Najath Akram, Viduneth Ariyaratna, Soumyajit Mandal, Vitor A. Coutinho, Fabio M. Bayer, Diego Coelho, and Theodore S. Rappaport. Fast radix-32 approximate dfts for 1024-beam digital rf beamforming. *IEEE Access*, 8, 2020.
- [16] Marco Mercuri, Pietro Russo, Miguel Glassee, Ivan Dario Castro, Eddy De Greef, Maxim Rykunov, Marc Bauduin, André Bourdoux, Ilja Ocket, Felice Crupi, and Tom Torfs. Automatic radar-based 2-d localization exploiting vital signs signatures. *Scientific Reports*, 12, MAY 2022.
- [17] Prateek Nallabolu, Li Zhang, Hong Hong, and Changzhi Li. Human presence sensing and gesture recognition for smart home applications with moving and stationary clutter suppression using a 60-ghz digital beamforming fmcw radar. *IEEE Access*, 9:72857–72866, 2021.

- [18] Novelda. *X7 Dev Kit User Guides*, 2023.
- [19] Rafael Padilla, Wesley L. Passos, Thadeu L. B. Dias, Sergio L. Netto, and Eduardo A. B. da Silva. A comparative analysis of object detection metrics with a companion open-source toolkit. *Electronics*, 10(3), 2021.
- [20] Massimo Pavan, Armando Caltabiano, and Manuel Roveri. On-device subject recognition in uwb-radar data with tiny machine learning. In *CPS Summer School, PhD Workshop*, 2022.
- [21] Massimo Pavan, Armando Caltabiano, and Manuel Roveri. Tinymml for uwb-radar based presence detection. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2022.
- [22] Massimo Pavan, Luis González Navarro, Armando Caltabiano, and Manuel Roveri. Unveiling the potential of tiny machine learning for enhanced people counting in uwb radar data. 2023.
- [23] James Raul. YOLOv5 transfer learning model for Edge Impulse, 2023.
- [24] Manuel Roveri. Is tiny deep learning the new deep learning? In Rajkumar Buyya, Susanna Munoz Hernandez, Ram Mohan Rao Kovvur, and T. Hitendra Sarma, editors, *Computational Intelligence and Data Analytics*, pages 23–39, Singapore, 2023. Springer Nature Singapore.
- [25] Christian A. Schroth, Christian Eckrich, Ibrahim Kakouche, Stefan Fabian, Oskar von Stryk, Abdelhak M. Zoubir, and Michael Muma. Emergency response person localization and vital sign estimation using a semi-autonomous robot mounted sfcw radar. *IEEE Transactions on Biomedical Engineering*, page 1–14, 2024.
- [26] Yizhou Wang, Zhongyu Jiang, Yudong Li, Jenq-Neng Hwang, Guanbin Xing, and Hui Liu. Rodnet: A real-time radar object detection network cross-supervised by camera-radar fused object 3d localization. *IEEE Journal of Selected Topics in Signal Processing*, 15(4):954–967, June 2021.
- [27] Zhong-Qiu Zhao, Peng Zheng, Shou tao Xu, and Xindong Wu. Object detection with deep learning: A review, 2019.

Abstract in lingua italiana

Nell'era della rapida evoluzione tecnologica, l'incremento di dispositivi embedded, dell'edge computing e delle unità IoT sta ridisegnando i paradigmi dell'informatica verso sistemi distribuiti ed edge. Il TinyML, o Tiny Machine Learning, si pone in prima linea in questa trasformazione, consentendo l'esecuzione di algoritmi di IA su dispositivi con risorse limitate. Aspetto particolarmente rilevante nel contesto dell'IoT e dell'edge computing, è la capacità del TinyML di facilitare l'elaborazione dei dati vicino alla loro fonte di acquisizione per ridurre la latenza, risparmiare energia e assicurare la privacy dei dati. I radar a banda ultralarga (UWB) sono una tecnologia particolarmente promettente per varie applicazioni tra cui la localizzazione, il rilevamento della presenza e la comunicazione, grazie alla loro ampiezza di banda, al costo relativamente basso, al basso consumo energetico e alle robuste prestazioni, rappresentando una soluzione all'avanguardia che si integra perfettamente con i principi del TinyML. In questa tesi introduciamo per la prima volta in letteratura, una soluzione TinyML leggera e rispettosa della privacy per la localizzazione umana in ambienti interni, tramite l'utilizzo di radar UWB a ingressi multipli e uscite multiple. Per raggiungere questo obiettivo proponiamo una pipeline che estrae mappe Distanza-Angolo da segnali di antenne riceventi UWB multiple e le utilizza come input per una rete leggera di rilevamento di oggetti, adattata per lavorare su dati radar per localizzare e contare persone. La nostra soluzione è stata testata su più set di dati con diverse configurazioni di antenne radar UWB, analizzando il trade-off tra i diversi livelli di consumo energetico e la capacità di localizzazione.

Parole chiave: Radar UltraWideBand; Immagini radar; Tiny Machine Learning; Localizzazione umana;